

SBT-DF203 - Lab 6:

Firewall Traffic Control

and Forensic Verification

Digital Forensics Investigation Report

Course Information

Field	Details
Course Code	SBT-DF203
Course Title	Basic Networking Skills for Digital Forensics
Lab Number	6
Lab Title	Firewall Traffic Control and Forensic Verification
Platform	Kali Linux Virtual Machine
Student	Adekunle Ogunyemi
Student ID	2025/FWSD/11479
Date	18 September 2026

Case Information

Item	Details
Case Reference	SBT-DF203-Lab6-Firewall-Verification
Investigator	Adekunle Ogunyemi
Evidence Type	Firewall rules, terminal output, network traffic, packet captures and firewall counters
Investigation Purpose	To verify how a source-specific firewall DROP rule affects HTTP traffic and to support the result with packet evidence and firewall counters

1 Executive Summary

The purpose of this laboratory exercise was to verify the effectiveness of a host-based firewall using iptables and to examine the resulting network traffic through forensic analysis. The lab involved establishing a normal HTTP connection between a client and a web server, applying a firewall rule to block HTTP traffic from a specific source IP address, capturing the resulting network traffic, and comparing the allowed and blocked communication patterns.

The results showed that HTTP traffic was initially successful before the firewall rule was applied. After the DROP rule was inserted into the INPUT chain, the specified client was unable to access the web service and experienced connection timeouts. Packet captures and firewall counters confirmed that the firewall rule was actively blocking the traffic. The rule was later removed and normal access was restored. These findings demonstrated the importance of firewall controls and the value of packet captures and rule counters as forensic evidence.

Key Findings

Finding	Result
Baseline HTTP connection	The client connected to the Apache server. The response was HTTP 404 Not Found, but the connection succeeded.
Firewall action	A source-specific DROP rule was inserted at the top of the INPUT chain.
Blocked connection	The client request timed out after about 10 seconds.
Packet evidence	The client SYN was visible, but the expected server response and completed HTTP session were not observed in the blocked test.

Firewall counter	The rule recorded 28 packets and 1,680 bytes.
Restoration	The DROP rule was removed, and the firewall configuration was checked.

2. Authority, Scope and Evidence Description

2.1 Authority

This work was carried out as part of the SBT-DF203 practical lab. The activity was performed in a controlled lab environment using systems assigned to the exercise.

2.2 Scope

The investigation was limited to the Kali Linux firewall configuration, the Apache web service, the client connection, packet captures and firewall counters generated during the test. No unrelated systems or external networks were included in the scope.

2.3 Evidence Description

Evidence Item	Description
E-01	Exported firewall ruleset before the test
E-02	Network interface and routing verification output
E-03	Apache listening-state verification
E-04	Baseline curl output showing successful connection and HTTP 404 response
E-05	Baseline packet capture showing TCP handshake and HTTP exchange
E-06	iptables DROP rule output for source 192.168.20.131 and TCP port 80
E-07	Blocked curl output showing timeout
E-08	Blocked traffic packet capture

E-09	Firewall counter output showing 28 packets and 1,680 bytes
E-10	Post-test firewall configuration after rule removal

3. Environment and Acquisition Method

3.1 Working Environment

The test used a Kali Linux virtual machine as the firewall and web server environment, together with a separate client system. Apache was used to provide the HTTP service on TCP port 80. Traffic was captured with Wireshark or tcpdump, while iptables commands were used to inspect and change the firewall rules.

```
(kali㉿kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab6/evidence]
$ ip -br address
lo                UNKNOWN          127.0.0.1/8 ::1/128
eth0              UP                192.168.20.129/24 fe80::d050:ecb7:752c:914e/64
docker0          DOWN             172.17.0.1/16
```

```
(kali㉿kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab6]
$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: disabled)
   Active: active (running) since Mon 2026-09-21 11:51:45 EDT; 21s ago
     Invocation: 4b25f375455a496e8290146e333095dd
       Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 70242 (apache2)
     Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served: 0"
      Tasks: 6 (limit: 2060)
    Memory: 29.4M (peak: 29.9M)
       CPU: 107ms
    CGroup: /system.slice/apache2.service
            └─70242 /usr/sbin/apache2 -k start -DFOREGROUND
              └─70245 /usr/sbin/apache2 -k start -DFOREGROUND
                └─70246 /usr/sbin/apache2 -k start -DFOREGROUND
```

3.2 Acquisition Method

The evidence was collected by recording the firewall rules before and after the test, running a baseline HTTP request, capturing the traffic, applying the DROP rule, repeating the request, and comparing the two results. The

evidence should be saved in the lab evidence folder with clear filenames and timestamps.

```
(kali@kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab6]
$ find . -maxdepth 1 -type d -print
.
./evidence
./reports
./scripts
./exported
./working
./screenshots

(kali@kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab6]
$ █
```

4. Evidence Integrity and Chain of Custody

Item	Collected By	Date/Time	Storage Location	Integrity Check
Firewall ruleset	Adekunle O.	8 Sept. 2026	evidence/	31c9b9f1bb901ce9bf700891a67f6aa4e2eae3b1e227a8778773
Baseline capture	Adekunle O.	8 Sept. 2026	evidence/	31c9b9f1bb901ce9bf700891a67f6aa4e2eae3b1e227a8778773
Blocked capture	Adekunle O.	8 Sept. 2026	evidence/	31c9b9f1bb901ce9bf700891a67f6aa4e2eae3b1e227a8778773
Terminal outputs	Adekunle O.	8 Sept. 2026	screenshots/ or evidence/	31c9b9f1bb901ce9bf700891a67f6aa4e2eae3b1e227a8778773

```
(kali@kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab6]
$ sha256sum iptables_before.rules
31c9b9f1bb901ce9bf700891a67f6aa4e2eae3b1e227a87787737e387c75bd78  iptables_before.rules

(kali@kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab6]
```

5. Part A – Baseline System and Firewall State

The firewall configuration was checked before any changes were made. The original ruleset was exported and saved. The system used UFW-managed

rules in the INPUT and FORWARD chains, and no custom HTTP blocking rule was present at the start of the test.

```
Session Actions Edit View Help
(kali㉿kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab6]
$ sudo iptables-save | tee reports/iptables_before.rules.txt
# Generated by iptables-save v1.8.13 (nf_tables) on Mon Sep 21 13:10:10 2026
*filter
:INPUT DROP [2:1270]
:FORWARD DROP [0:0]
:OUTPUT ACCEPT [8:320]
:DOCKER - [0:0]
:DOCKER-BRIDGE - [0:0]
:DOCKER-CT - [0:0]
:DOCKER-FORWARD - [0:0]
:DOCKER-ISOLATION-STAGE-1 - [0:0]
:DOCKER-ISOLATION-STAGE-2 - [0:0]
:DOCKER-USER - [0:0]
:ufw-after-forward - [0:0]
:ufw-after-input - [0:0]
:ufw-after-logging-forward - [0:0]
:ufw-after-logging-input - [0:0]
:ufw-after-logging-output - [0:0]
:ufw-after-output - [0:0]
:ufw-before-forward - [0:0]
:ufw-before-input - [0:0]
:ufw-before-logging-forward - [0:0]
:ufw-before-logging-input - [0:0]
:ufw-before-logging-output - [0:0]
:ufw-before-output - [0:0]
:ufw-logging-allow - [0:0]
:ufw-logging-deny - [0:0]
:ufw-not-local - [0:0]
:ufw-reject-forward - [0:0]
:ufw-reject-input - [0:0]
```

The client successfully connected to the web server and received an HTTP response. This confirmed that communication between the client and server was functioning correctly before firewall restrictions were applied.

```
solutions@solutions-VMware-Virtual-Platform:~/Desktop/Adekunle_Ogunyemi$ sudo curl -v --connect-timeout 5 http://192.168.20.129/24/firewall_lab.html 2>&1 | tee baseline_curl.txt
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           %             %             Dload  Upload  Total   Spent    Left   Speed
0      0     0     0     0     0      0     0  --:--:--  --:--:--  --:--:--    0*
Trying 192.168.20.129:80...
* Connected to 192.168.20.129 (192.168.20.129) port 80
> GET /24/firewall_lab.html HTTP/1.1
> Host: 192.168.20.129
> User-Agent: curl/8.5.0
> Accept: */*
>
< HTTP/1.1 404 Not Found
< Date: Mon, 21 Sep 2026 17:41:22 GMT
< Server: Apache/2.4.68 (Debian)
< Content-Length: 316
< Content-Type: text/html; charset=iso-8859-1
<
{ [316 bytes data]
100  316  100  316    0     0  41573      0  --:--:--  --:--:--  --:--:--  45142
```

This demonstrates that communication existed before firewall restrictions were applied.

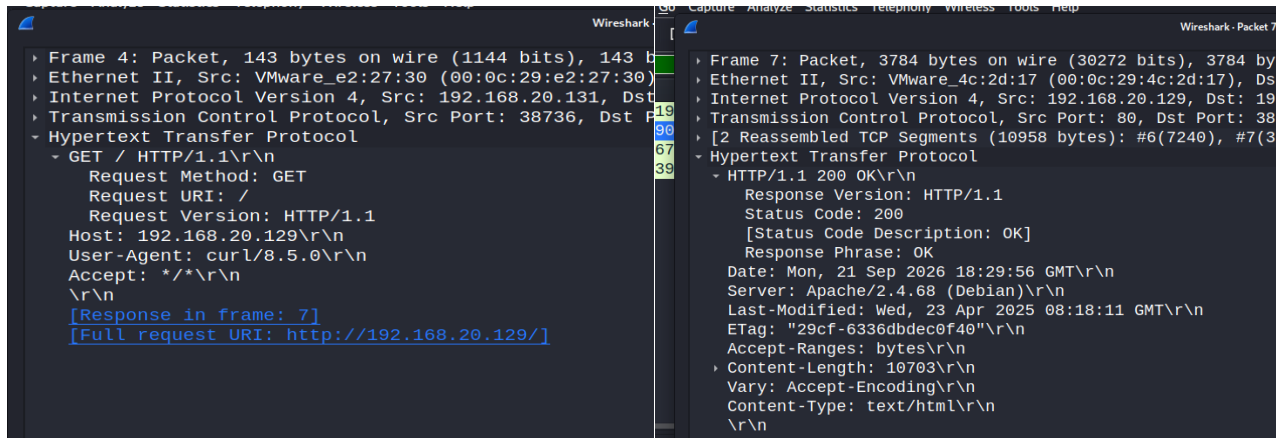
The network interface, IP address and routing information were also recorded before the traffic tests.

6. Part B – Baseline HTTP Connection

The client first connected to the Apache server without the new blocking rule. The curl test connected successfully. The server returned HTTP 404 Not Found, but this did not prevent the TCP connection from being established. This result was used as the baseline for comparison with the blocked test.

No.	Time	Delta	Source	Destination	Protocol	Length	Info
1	0.000000000		192.168.20.131	192.168.20.129	TCP	74	38736 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=
2	0.000778114	0.000778114	192.168.20.129	192.168.20.131	TCP	74	80 → 38736 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK
...	10.411186093	10.410407979	192.168.20.131	91.189.91.57	TCP	74	47418 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=
...	10.606732040	0.195545947	91.189.91.57	192.168.20.131	TCP	60	80 → 47418 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460

The baseline packet capture showed the normal TCP exchange: SYN from the client, SYN-ACK from the server, ACK from the client, followed by the HTTP request and response.



7. Part C – Firewall Rule Implementation

A source-specific DROP rule was added for client IP address 192.168.20.131. The rule applied to TCP traffic going to destination port 80. It was inserted at the top of the INPUT chain so that it was processed before the existing UFW rules.

Rule logic: traffic from 192.168.20.131 to TCP port 80 was silently discarded. DROP was used instead of REJECT so that the firewall would not send an immediate error response to the client.


```
(kali㉿kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab6]
$ sudo iptables -I INPUT 1 -s 192.168.20.131 -p tcp --dport 80 -j DROP

(kali㉿kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab6]
$ sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_add.tx
Chain INPUT (policy DROP 2 packets, 1270 bytes)
num  pkts bytes target     prot opt in     out     source         destination
1      0      0 DROP      tcp  --  *      *       192.168.20.131  0.0.0.0/0
p dpt:80
2      0      0 DROP      tcp  --  *      *       192.168.56.20   0.0.0.0/0
p dpt:80
3  926K 1327M ufw-before-logging-input  all  --  *      *       0.0.0.0/0
0
4  926K 1327M ufw-before-input          all  --  *      *       0.0.0.0/0      0.0.0.0/0
5    41 11733 ufw-after-input           all  --  *      *       0.0.0.0/0      0.0.0.0/0
6     2  1270 ufw-after-logging-input    all  --  *      *       0.0.0.0/0
```

8. Part D – Blocked HTTP Test

After adding the rule, the same client request was repeated. The curl command did not receive a response and timed out after about 10 seconds. No HTTP session was completed during the blocked test. This behaviour was consistent with a DROP rule because the matching packets were discarded without an immediate rejection message.

```
solutions@solutions-VMware-Virtual-Platform:~/Desktop/Adekunle_Ogunyemi$ curl -v
--connect-timeout 10 http://192.168.20.129/firewall_lab.html
* Trying 192.168.20.129:80...
* ipv4 connect timeout after 10000ms, move on!
* Failed to connect to 192.168.20.129 port 80 after 10003 ms: Timeout was reached
* Closing connection
curl: (28) Failed to connect to 192.168.20.129 port 80 after 10003 ms: Timeout was reached
solutions@solutions-VMware-Virtual-Platform:~/Desktop/Adekunle_Ogunyemi$
```

The blocked packet capture showed the client attempting to start the connection, but the expected server response was not seen and the handshake did not complete. The absence of a completed handshake and HTTP exchange supported the blocked-traffic finding.

9. Part E – Firewall Counter and Packet Correlation

The firewall counter for the DROP rule increased during the blocked test. The recorded value was 28 packets and 1,680 bytes. This counter supported the conclusion that packets matched the rule.

```
(kali㉿kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab6]
$ sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_restored.txt
Chain INPUT (policy DROP 2 packets, 1270 bytes)
num  pkts bytes target     prot opt in     out     source        destination
1      0      0 DROP       tcp  --  *      *        192.168.56.20  0.0.0.0/0
p dpt:80
2    928K 1328M ufw-before-logging-input  all  --  *      *        0.0.0.0/0
0
3    928K 1328M ufw-before-input          all  --  *      *        0.0.0.0/0      0.0.
4      48 13336 ufw-after-input           all  --  *      *        0.0.0.0/0      0.0.
5       2  1270 ufw-after-logging-input    all  --  *      *        0.0.0.0/0
6       2  1270 ufw-reject-input           all  --  *      *        0.0.0.0/0      0.0
```

Observation	Allowed Baseline	Blocked Test
Client SYN visible	Yes	Yes
Server SYN-ACK visible	Yes	No response observed
TCP handshake completed	Yes	No
HTTP request/response	Yes	No
Retransmissions	Minimal	Present
Firewall counter increase	No new DROP match	Yes
Curl result	Connection completed	Timeout

10. Part F – DROP and REJECT Comparison

Action	Behaviour	Expected Client Result
DROP	Silently discards matching packets and does not send an immediate error response.	The client may retry and eventually time out.
REJECT	Rejects the traffic and sends an error response, depending on the reject type.	The client usually fails faster and may report a connection refusal or reset.

The observed timeout in this lab was consistent with DROP. The exact client message can vary depending on the operating system, TCP state and the type of traffic being tested.

11. Part G – Rule Removal and Restoration

After the blocked test was completed, the DROP rule was removed. The firewall configuration was then checked to confirm that the temporary rule was no longer present.

```
(kali㉿kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab6]
$ sudo iptables -D INPUT -s 192.168.20.131 -p tcp --dport 80 -j DROP

(kali㉿kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab6]
$
```



```
Session Actions Edit View Help
kali@kali: ~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab6  kali@kali: ~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab6/evidence
(kali㉿kali)-[~/Desktop/Adekunle_Ogunyemi/SBT-DF203-Lab6]
$ sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_restored.txt

Chain INPUT (policy DROP 4 packets, 2540 bytes)
num  pkts bytes target     prot opt in     out     source        destination
1      0    0 DROP      tcp  --  *      *       192.168.56.20  0.0.0.0/0
    tcp dpt:80
2   951K 1353M ufw-before-logging-input all  --  *      *       0.0.0.0/0     0.0.0.0/0
3   951K 1353M ufw-before-input  all  --  *      *       0.0.0.0/0     0.0.0.0/0
4     90 23336 ufw-after-input   all  --  *      *       0.0.0.0/0     0.0.0.0/0
5      4  2540 ufw-after-logging-input all  --  *      *       0.0.0.0/0     0.0.0.0/0
6      4  2540 ufw-reject-input  all  --  *      *       0.0.0.0/0     0.0.0.0/0
7      4  2540 ufw-track-input   all  --  *      *       0.0.0.0/0     0.0.0.0/0
```

Removing a rule by line number should be done carefully because line numbers can change after rules are inserted or deleted. A rule should be confirmed before deletion, and the final configuration should be saved as evidence.

12. Required Forensic Findings

Question	Finding
Why did DROP cause SYN retransmissions and a timeout?	The firewall silently discarded matching packets. The client did not receive the expected response, so it retried the connection and eventually timed out.
How would REJECT differ from DROP?	REJECT sends an error response instead of silently discarding the traffic. The connection normally fails faster.
What do the firewall counters prove?	The increase to 28 packets and 1,680 bytes shows that traffic matched the DROP rule. The counter alone does not prove the full content of every packet.
How can a blocked service be distinguished from a stopped web server?	The investigator should compare the listening state, firewall counters and packet capture. A stopped service may show no listener and may produce a connection-refused response when no firewall rule blocks it. In this test, the matching DROP counter and the blocked packet behaviour supported firewall blocking.
What is the risk of deleting by line number?	Line numbers can change when rules are added or removed. The wrong rule may be deleted if the current rule order is not checked first.

13. Limitations and Challenges

The test was carried out in a controlled lab environment, so the result may not represent every firewall or network setup. The HTTP response in the baseline test was 404 Not Found, which confirms that the connection reached the service but does not confirm that the requested page existed. The packet capture and firewall counter should be interpreted together because a counter shows matched traffic but does not provide all packet details. All final conclusions should be supported by the original capture files, terminal outputs and screenshots.

14. Conclusion

This lab demonstrated how a source-specific iptables DROP rule can block HTTP traffic from a selected client. The baseline test showed a working TCP and HTTP connection. After the DROP rule was added for source IP 192.168.20.131 and TCP port 80, the client timed out, the handshake did not complete and the firewall counter increased to 28 packets and 1,680 bytes. The packet capture and firewall output supported the conclusion that the traffic was being blocked by the firewall. The temporary rule was removed after testing and the configuration was checked.

15. Recommendations

- ❖ Save the original and final firewall rulesets for comparison.
- ❖ Record the exact date, time, interface and IP addresses for every capture.
- ❖ Hash all evidence files after collection and record the values in the evidence log.
- ❖ Use clear filenames for baseline and blocked captures.
- ❖ Check firewall counters before and after each test.
- ❖ Avoid deleting firewall rules by line number without first confirming the current rule order.
- ❖ Keep screenshots readable and include the command and output where possible.

Appendix – Academic Integrity Statement

I confirm that this report describes the work completed in the assigned practical lab. The screenshots, command outputs, capture files and hash values should be inserted from my own lab session. Any assistance used in preparing the report should not replace the actual evidence collected during the exercise.